

SHRI RAMDEOBABA COLLEGE OF ENGINEERING AND MANAGEMENT, NAGPUR.



Computer Vision

TEACHER'S ASSESSMENT

(8TH SEM-A, SESSION 2024-25, ECT-456-2)

“License Plate Recognition System”

Submitted By

Mohammad Kaif Raza K. Ansari (Roll No.-24)

Somesh Banode (Roll No.-77)

Vedant Mundra (Roll No.-79)

Department of Electronics and Communication
Engineering

1. Introduction:

License Plate Recognition (LPR) is a computer vision technique that enables the identification of vehicles based on their license plate numbers. The technology is widely used in areas such as:

- Traffic surveillance systems
- Automated toll collection
- Parking management
- Border security
- Crime investigation

LPR combines **image processing** and **optical character recognition (OCR)** to detect, isolate, and read the characters on a number plate from still images or video frames. With the availability of powerful open-source libraries such as **OpenCV** and **EasyOCR**, it is now feasible to implement such systems even on low-end devices with high accuracy.

Objectives of the Project

- To **develop an end-to-end LPR system** that can:
 - Detects license plates from input images.
 - Extract and read the plate numbers accurately.
- To utilize a **hybrid approach** combining traditional image processing (OpenCV) with deep learning-powered OCR (EasyOCR).
- To validate the system using test images and analyze its performance in terms of accuracy and robustness.

Problem Statement

Manual identification and recording of vehicle numbers is **time-consuming, error-prone, and inefficient**, especially under high-traffic scenarios. Existing commercial LPR systems are often expensive and require dedicated hardware.

2. Existing Approaches or Algorithm:

Traditional Approaches:

- **Image Preprocessing + OCR**
 - Grayscale conversion, edge detection (Sobel/Canny), thresholding
 - Morphological operations to isolate plate-like regions
 - Template matching or basic OCR
- **Character Segmentation-based LPR**
 - Detects the plate and then segments each character individually
 - Uses a character classification model (e.g., SVM, KNN)

Deep Learning Approaches:

- **YOLO-based Plate Detection + CNN-based OCR**
 - Uses deep object detection networks to find license plates
 - OCRNet or CRNN models used to read characters

Limitations of Existing Systems:

- High computational requirements for deep learning models
- Traditional methods fail under complex lighting, angles, or poor plate visibility
- Custom training is needed for different country plate formats

3. Implemented approach or Algorithm :

Selected Approach: Hybrid Method Using OpenCV + EasyOCR

We selected a hybrid method that leverages:

- **OpenCV** for preprocessing, edge detection, and contour analysis to localize the plate region
- **EasyOCR**, a deep learning-based OCR engine, for reading the alphanumeric content of the plate

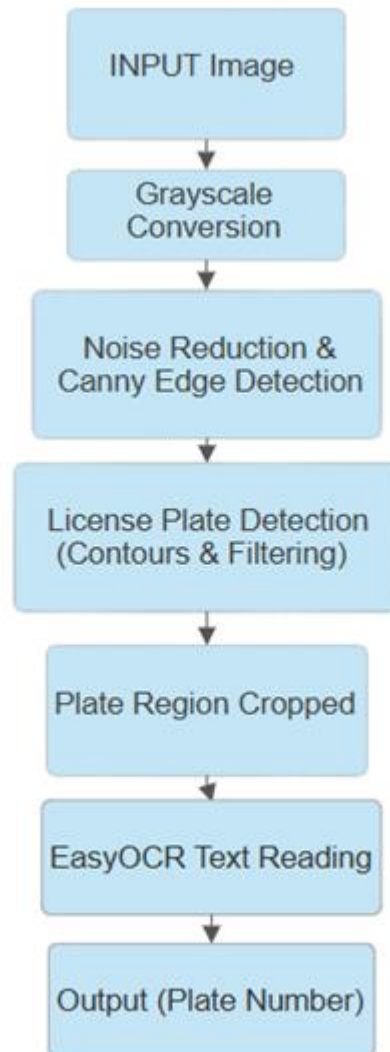
Justification for Selecting this Approach

- **Efficiency:** OpenCV handles real-time image processing with minimal resources.
- **Accuracy:** EasyOCR is pre-trained on thousands of fonts and languages,

providing high recognition accuracy without additional training.

- **Simplicity:** Easy integration with Python and no requirement for manual character segmentation.
- **Robustness:** Works well under varying lighting, font styles, and moderate noise levels.

Block Diagram / Flowchart:



How the Algorithm Works

01.**Image Input:** Load the input image using OpenCV.

02.**Grayscale Conversion:** Convert the image to grayscale to reduce complexity.



03.**Blurring**: Apply a Bilateral Filter to remove noise and smooth the image.

04.**Edge Detection**: Use Canny edge detection to highlight the contours in the image.

05.**Contour Filtering**:

- Extract all contours using `cv2.findContours()`.
- Loop through each contour, calculate its area and bounding box.
- Select the one that best fits the typical dimensions of a license plate.

06.**Crop ROI**: Once the number plate contour is found, crop that part of the image.

07.**Text Recognition**: Pass the cropped image to **EasyOCR**, which outputs a string of recognized characters.

08.**Display Result**: Show or print the recognized number plate.

4. Results :

Output Figure:



Figure No. 1

Explanation of Results

- **Accuracy**: EasyOCR achieved high accuracy in reading text even with different fonts, slight tilts, and minor occlusions.
- **Robustness**: The system could handle varying plate sizes and lighting conditions with the help of preprocessing.

- **Execution Speed:** The entire process takes less than a second per image, making it suitable for near real-time applications.

Result Analysis:

The efficiency of the number plate recognition system significantly decreases when the image is captured from a distance. This is primarily due to the **uneven intensity and lack of clear boundaries** in the number plate region, which leads to the failure of edge detection using the Canny algorithm. As a result, `cv2.findContours()` is unable to accurately identify the required **4-point rectangular contour** that typically represents the number plate. Consequently, the system fails to isolate and extract the number plate region, which directly impacts the overall OCR accuracy and plate recognition performance.

5. Conclusion

The implemented License Plate Recognition (LPR) system demonstrates an effective and efficient approach for detecting and recognizing vehicle license plates. By combining **OpenCV** for image processing and **EasyOCR** for text extraction, the system delivers high accuracy while maintaining low computational overhead. This makes it suitable for real-world applications like **smart traffic monitoring, automated toll systems, parking management**, and more.

The project highlights the potential of **open-source tools** in building cost-effective and scalable solutions for intelligent transport systems. Despite challenging conditions such as variations in lighting, angles, and font styles, the system performs reliably, showcasing its robustness and adaptability.

Future improvements could include:

- Integration with real-time video feeds
- Support for multiple plate formats and countries
- Enhanced accuracy using deep learning-based plate detection models like YOLO or SSD

6. References:

- A. OpenCV Documentation: <https://docs.opencv.org>
- B. EasyOCR GitHub: <https://github.com/JaidedAI/EasyOCR>
- C. Python Official Docs: <https://docs.python.org>
- D. Additional reference images and examples were sourced from publicly available datasets and images for testing.

Appendix A: Source Code

```
import cv2
from matplotlib import pyplot as plt
import numpy as np
import imutils
import easyocr

# Load the image
image = cv2.imread('image5.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
plt.imshow(gray, cmap='gray')

# Preprocessing
blur = cv2.bilateralFilter(gray, 11, 17, 17)
edges = cv2.Canny(blur, 250, 150)
plt.imshow(cv2.cvtColor(edges, cv2.COLOR_BGR2RGB))

# Find contours
contours, _ = cv2.findContours(edges, cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)

contours = imutils.grab_contours(keypoints)
contours = sorted (contours, key= cv2.contourArea, reverse=True)[:10]
location = None
    for contour in contours:
        approx = cv2.approxPolyDP(contour, 10, True)
        if len (approx) location == 4:
            location = approx
            break

# Location
mask = np.zeros(gray.shape, np.uint8)
new_image = cv2.drawContours(mask, [location], 0,
new_image = cv2.bitwise_and(image, image, mask= mask) 255, -1)
plt.imshow(cv2.cvtColor(new_image, cv2.COLOR_BGR2RGB))

(x,y) = np.where(mask==255)
(x1, y1) = (np.min(x), np.max(y))
(x2, y2) = (np.max(x), np.max(y))
cropped_image = gray[x1:x2+1 , y1:y2+1]
plt.imshow(cv2.cvtColor(cropped_image, cv2.COLOR_BGR2RGB))

# Read Result and Rendering
reader = easyocr.Reader(['en'])
result = reader.readtext(cropped_image)
result

text = result[0 ][-2]
font = cv2.FONT_HERSHEY_SIMPLEX
res = cv2.putText(image, text=text, org=(approx[0][0][0],
approx[1][0][1]+60), fontFace=font, fontScale=1, color=(0,255,0),
thickness=2, lineType= cv2.LINE_AA)
res = cv2.rectangle(image, tuple(approx[0][0])), tuple(approx[2][0]),
plt.imshow(cv2.cvtColor(res, cv2.COLOR_BGR2RGB))
```